

使用 Node.JS 和 Google Cloud Functions 觸發 DAG

來源：<https://codelabs.developers.google.com/codelabs/triggering-dag-gcf?hl=zh-tw>

步驟數：7

本程式碼研究室說明如何在 Google Cloud Composer 中使用 Google Cloud Functions，以觸發 Apache Airflow 工作流程 (DAG)。這裡的 DAG 會使用 BashOperator 執行簡單的 bash 指令

目錄

1. [1. 簡介](#)
2. [2. 設定 GCP](#)
3. [3. 設定 Google Cloud Functions \(GCF\)](#)
4. [4. 建立函式](#)
5. [5. 設定 DAG](#)
6. [6. 測試函式](#)
7. [7. 清除所用資源](#)

1. 簡介

Apache Airflow 的設計是定期執行 DAG，但您也可以發生事件時觸發 DAG，例如當 Cloud Storage 值區有所異動或有訊息推送至 Cloud Pub/Sub 時。為此，您可以使用 [Cloud Functions](#) 觸發 Cloud Composer DAG。

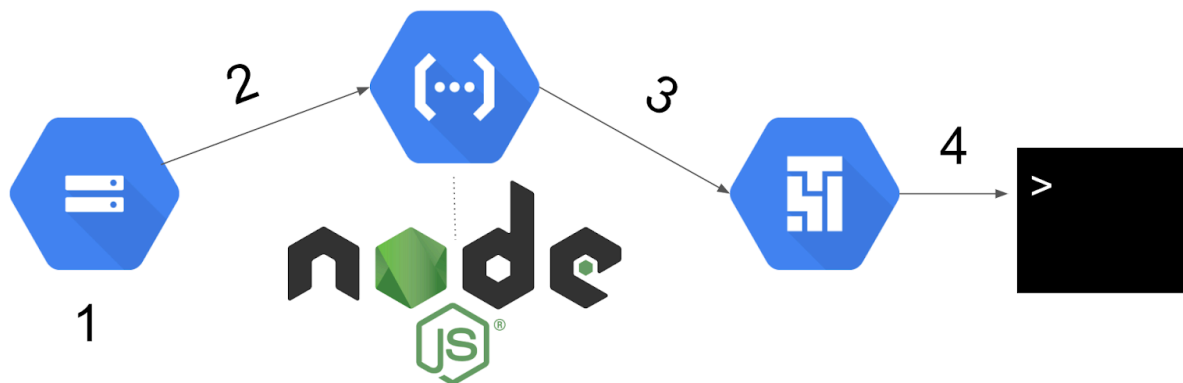
本實驗室的範例程式會在 Cloud Storage bucket 每次有所異動時執行簡單的 DAG。這個 DAG 會使用 BashOperator 執行 bash 指令，列印上傳至 Cloud Storage bucket 的內容異動資訊。

開始本實驗室前，建議先完成「[Cloud Composer 簡介](#)」和「[開始使用 Cloud Functions](#)」程式碼研究室。如果您在「Cloud Composer 簡介」程式碼研究室中建立 Composer 環境，可以在本實驗室中使用該環境。

建構項目

本程式碼研究室涵蓋下列內容：

1. 將檔案上傳至 [Google Cloud Storage](#)，
2. 使用 Node.JS 執行階段觸發 [Google Cloud Function](#)
3. 這項函式會在 [Google Cloud Composer](#) 中執行 DAG
4. 這會執行簡單的 Bash 指令，將變更內容列印至 Google Cloud Storage bucket



課程內容

- 如何使用 Google Cloud Functions + Node.js 觸發 Apache Airflow DAG

事前準備

- GCP 帳戶
 - 對 JavaScript 有基本瞭解
 - Cloud Composer/Airflow 和 Cloud Functions 的基本知識
 - 熟悉使用 CLI 指令
-

步驟 2 · 約 5 分鐘

2. 設定 GCP

選取或建立專案

選取或建立 Google Cloud Platform 專案。如要建立新專案，請按照[這裡](#)的步驟操作。

記下專案 ID，後續步驟將會用到。

如要建立新專案，專案 ID 會顯示在建立頁面的「專案名稱」下方

New Project

Project Name *

My Project 303

Project ID: plucky-avatar-233921. It cannot be changed later. [EDIT](#)

Billing account *

My Billing Account

Any charges for this project will be billed to the account you select here.

Organization

google.com

This project will be attached to google.com.

Location *

[BROWSE](#)

Parent organization or folder

☐ Create Googleplex internal App Engine project ?

[CREATE](#)

[CANCEL](#)

如果已建立專案，您可以在[控制台首頁](#)的「專案資訊」資訊卡中找到 ID

[DASHBOARD](#) [ACTIVITY](#) [CUSTOMIZE](#)

Quick Access ([Feedback?](#))



Cloud Storage Browser

Accessed 2 minutes ago



Environments -
Composer - My Project...

Accessed 3 minutes ago



Composer - My Project
56570 - Google Cloud...

Accessed 3 minutes ago



Manage Resources

Accessed 1 hour ago



Project info

Project name

My Project 56570

Project ID

calcium-post-233919

Project number

765545603813

[Go to project settings](#)



Resources

This project has no resources



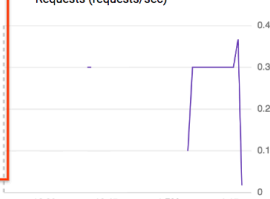
Trace

No trace data from the past 7 days

[Get started with Stackdriver Trace](#)

APIs

Requests (requests/sec)



Requests: 0.017

[Go to APIs overview](#)



Google Cloud Platform status

All services normal

[Go to Cloud status dashboard](#)

Billing

Estimated charges USD \$0.00

For the billing period Mar 1 - 8, 2019

[View detailed charges](#)



Error Reporting

No sign of any errors. Have you set up Error Reporting?

[Learn how to set up Error Reporting](#)

注意：如果您不打算保留在本教學課程中建立的資源，請建立新的專案，不要選擇現有的專案。這樣在課程結束後，您就可以刪除專案，移除與專案和教學課程相關聯的所有資源。

啟用 API

啟用 [Cloud Composer](#)、[Google Cloud Functions](#)、[Cloud Identity](#) 和 [Google Identity and Access Management \(IAM\) API](#)。

Register your application for Cloud Composer API, Cloud Functions API, Identity and Access Management (IAM) API in Google Cloud Platform

Google Cloud Platform allows you to manage your application and monitor API usage.

Select a project where your application will be registered
You can use one project to manage all of your applications, or you can create a different project for each application.

Create a project ▼

Continue

如要進一步瞭解如何啟用及停用 API，請參閱[說明文件](#)。

建立 Composer 環境

如果您已有 Cloud Composer 環境，請直接跳至下一節。

建立 [Cloud Composer 環境](#)，並採用下列設定：

- 名稱：my-composer-environment
- 位置：地理位置距離您最近的位置
- 區域：該地區中的任何可用區

其他設定則保留預設值。按一下底部的「建立」。記下 Composer 環境名稱和位置，後續步驟會用到。

 **Composer**

Environments

[+ CREATE](#)

Filter environments

☐	☐	Name ↑	Location	Creation time	Update t
No rows to display					

Composer

Create environment

Name *

my-composer-environment

Node configuration

The configuration information for the Google Kubernetes Engine nodes running the Airflow software.

Node count *

3

The number of nodes in the Google Kubernetes Engine cluster that will be used to run this environment.

Location *

us-central1

The Google Compute Engine region where the environment will be created.

Zone

us-central1-a

The Compute Engine zone in which to deploy the VMs used to run the Apache Airflow software. If unspecified, the service will pick a zone in the Compute Engine region corresponding to the selected location. [Learn more.](#)

如果在 GCP 主控台的「Environments」(環境) 頁面，環境名稱左側顯示綠色勾號，表示環境已建立完成。建立環境需要幾分鐘才能完成。在這段期間內，你仍可離開這個頁面。

建立 Cloud Storage bucket

在專案中建立 [Cloud Storage bucket](#)，並採用下列設定：

- 名稱：<your-project-id>
- 預設儲存空間級別：多區域
- 位置：地理位置最接近您使用的 Cloud Composer 區域的[位置](#)
- 存取權控管模型：設定物件層級和值區層級權限

準備就緒後，請按下「建立」。請務必記下 Cloud Storage bucket 的名稱，以供後續步驟使用。

Create a bucket

Name ?

Must be unique across Cloud Storage. If you're [serving website content](#), enter the website domain as the name.

calcium-post-233919

Default storage class

Objects added to this bucket are assigned the selected storage class by default. An object's storage class and bucket location affect its geo-redundancy, availability, and costs. You can set storage classes for individual objects in gsutil. [Learn more](#)

?

Nearline and Coldline data in multi-regional locations is now stored geo-redundantly. New locations nam4 and eur4 (available in beta) enable co-location of compute and storage for high performance with geo-redundancy. [Learn more](#)

Dismiss

☒ Multi-Regional

☐ Regional

☐ Nearline

☐ Coldline

Location

United States

Compare storage classes

Storage cost	Retrieval cost	Class A operations ?	Class B operations ?
\$0.026 per GB-month	Free	\$0.005 per 1,000 ops	\$0.0004 per 1,000 ops

Access control model

Choose how you'll control access to this bucket's objects. [Learn more](#)

☐ Set permissions uniformly at bucket-level (Bucket Policy Only)

Enforces the bucket's IAM policy without object ACLs. May help prevent unintended access. If selected, this option becomes permanent after 90 days.

☒ Set object-level and bucket-level permissions

Enforces the IAM policy and object ACLs for more granular control of object access.

Show advanced settings

Create

Cancel

步驟 3 · 約 5 分鐘

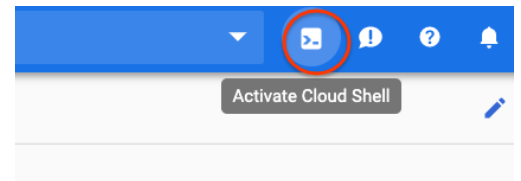
3. 設定 Google Cloud Functions (GCF)

如要設定 GCF，我們將在 Google Cloud Shell 中執行指令。

雖然您可以使用 `gcloud` 指令列工具，從筆電遠端操作 Google Cloud，但在本程式碼研究室中，我們將使用 [Google Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

如要啟用 Google Cloud Shell，請在開發人員控制台中按一下右上方的按鈕 (佈建並連線至環境的作業需要一些時間才能完成)：



為 Cloud Functions 服務帳戶授予 BLOB 簽署權限

如要讓 GCF 向保護 Airflow 網路伺服器的 Proxy [Cloud IAP](#) 進行驗證，您必須將 `Service Account Token Creator` 角色授予 Appspot 服務帳戶 GCF。方法是在 Cloud Shell 中執行下列指令，並將 `<your-project-id>` 替換為專案名稱。

```
gcloud iam service-accounts add-iam-policy-binding \  
<your-project-id>@appspot.gserviceaccount.com \  
--member=serviceAccount:<your-project-id>@appspot.gserviceaccount.com \  
--role=roles/iam.serviceAccountTokenCreator
```

舉例來說，如果專案名稱為 `my-project`，則指令應為

```
gcloud iam service-accounts add-iam-policy-binding \  
my-project@appspot.gserviceaccount.com \  
--member=serviceAccount:my-project@appspot.gserviceaccount.com \  
--role=roles/iam.serviceAccountTokenCreator
```

取得用戶端 ID

重要事項：如果 Composer 環境尚未建立完成，您將無法取得用戶端 ID。建立期間，您可以略過「建立函式」的「逐步執行程式碼」一節，進一步瞭解執行這個函式時會發生什麼情況。

如要建構權杖，藉此向 [Cloud IAP](#) 進行驗證，您必須為函式提供保護 Airflow 網路伺服器的 Proxy 用戶端 ID。Cloud Composer API 不會直接提供這項資訊，而會向 Airflow 網路伺服器發出未經授權的要求，並從重新導向網址中擷取用戶端 ID。我們將使用 Cloud Shell 執行 Python 檔案，擷取用戶端 ID。

在 Cloud Shell 中執行下列指令，從 GitHub 下載必要程式碼


```
cd
git clone https://github.com/GoogleCloudPlatform/python-docs-samples.git
```

如果收到錯誤訊息，指出這個目錄已存在，請執行下列指令，將目錄更新至最新版本

```
cd python-docs-samples/
git pull origin master
```

執行下列指令，切換至適當的目錄：

```
cd python-docs-samples/composer/rest
```

執行 Python 程式碼來取得用戶端 ID，並將專案名稱替換為 `<your-project-id>`，先前建立的 Composer 環境位置替換為 `<your-composer-location>`，先前建立的 Composer 環境名稱替換為 `<your-composer-environment>`

```
python3 get_client_id.py <your-project-id> <your-composer-location> <your-composer-environment>
```

舉例來說，如果專案名稱為 `my-project`、Composer 位置為 `us-central1`，且環境名稱為 `my-composer`，則指令會是

```
python3 get_client_id.py my-project us-central1 my-composer
```

`get_client_id.py` 會執行以下動作：

- 透過 Google Cloud 進行驗證
- 向 Airflow 網路伺服器發出未經授權的 HTTP 要求，以取得重新導向 URI
- 從該重新導向中擷取 `client_id` 查詢參數
- 並列印出來供你使用

指令列會列印出用戶端 ID，如下所示：

```
12345678987654321-abcdef3ghi5jkl7mno8pqr0.apps.googleusercontent.com
```

重要事項：請將用戶端 ID 複製到您可以在「建立函式」步驟中使用的位置，否則您稍後必須重複這個步驟。

步驟 4 · 約 0 分鐘

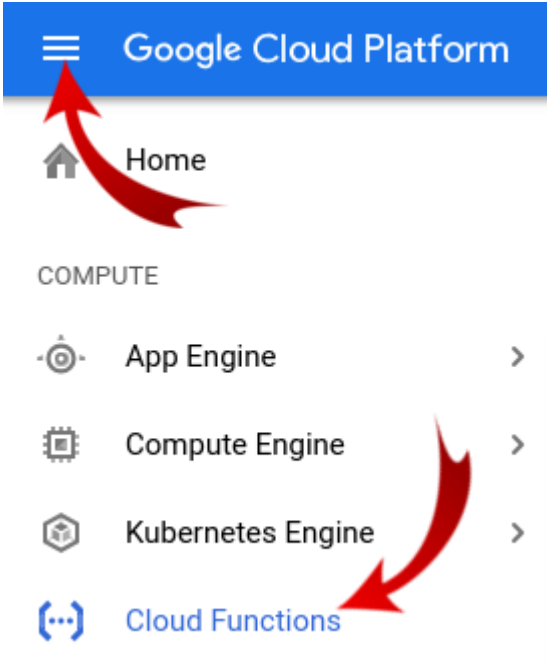
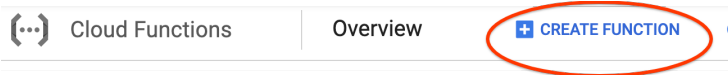
4. 建立函式

在 Cloud Shell 中執行下列指令，複製包含必要程式碼範例的存放區：

```
cd
git clone https://github.com/GoogleCloudPlatform/nodejs-docs-samples.git
```

變更至必要目錄，並在完成後續幾個步驟時，將 Cloud Shell 保持開啟

```
cd nodejs-docs-samples/composer/functions/composer-storage-trigger
```

依序點選「導覽選單」和「Cloud Functions」，前往 Google Cloud Functions 頁面	
按一下頁面頂端的「CREATE FUNCTION」(建立函式)	
將函式命名為「my-function」，並保留預設記憶體大小 256 MB。	

將觸發條件設為「Cloud Storage」，將事件類型保留為「Finalize/Create」，然後瀏覽至您在「建立 Cloud Storage bucket」步驟中建立的 bucket。	Trigger Cloud Storage Event Type ? Finalize/Create Bucket  my-project-241320 Browse
將「原始碼」設為「內嵌編輯器」，並將執行階段設為「Node.js 8」	Source code ☒ Inline editor ☐ ZIP upload ☐ ZIP from Cloud Storage ☐ Cloud Source repository Runtime Node.js 8

在 Cloud Shell 中執行下列指令。這會在 [Cloud Shell 編輯器](#) 中開啟 index.js 和 package.json

```
cloudshell edit index.js package.json
```

按一下 package.json 分頁標籤，複製該程式碼並貼到 Cloud Functions 內嵌編輯器的 package.json 區段	index.js <u>package.json</u> <pre> 1 { 2 "name": "nodejs-docs-samples-functions-composer-storage-trigger", 3 "version": "0.0.1", 4 "dependencies": { 5 "form-data": "^2.3.2", 6 "node-fetch": "^2.2.0" 7 }, 8 "engines": { 9 "node": ">=4.3.2" 10 }, 11 "private": true, 12 "license": "Apache-2.0", 13 "author": "Google Inc.", 14 "repository": { 15 "type": "git", 16 "url": "https://github.com/GoogleCloudPlatform/nodejs-docs-samples" 17 }, 18 "devDependencies": { 19 "@google-cloud/nodejs-repo-tools": "^2.2.5", 20 "ava": "0.25.0", 21 "proxyquire": "2.0.0", 22 "semistandard": "^12.0.1", 23 "sinon": "4.4.2" 24 }, 25 "scripts": { 26 "start": "node index.js" 27 } 28 }</pre>
將「要執行的函式」設為 triggerDag	Function to execute ? triggerDag

按一下 <code>index.js</code> 分頁標籤，複製程式碼，然後貼到 Cloud Functions 內嵌編輯器的 <code>index.js</code> 區段	index.jspackage.json <pre> 1 /** 2 * Copyright 2018 Google LLC 3 * 4 * Licensed under the Apache License, Version 2.0 (the "License"); 5 * you may not use this file except in compliance with the License. 6 * You may obtain a copy of the License at 7 * 8 * https://www.apache.org/licenses/LICENSE-2.0 9 * 10 * Unless required by applicable law or agreed to in writing, software 11 * distributed under the License is distributed on an "AS IS" BASIS, 12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either expressed or implied. 13 * See the License for the specific language governing permissions and 14 * limitations under the License. 15 */ 16 17 // [START composer_trigger] 18 'use strict'; 19 20 const fetch = require('node-fetch'); 21 const FormData = require('form-data'); 22 23 /** 24 * Triggered from a message on a Cloud Storage bucket. 25 * 26 * @param {Object} data The Cloud Storage bucket message. 27 */ </pre>
將 <code>PROJECT_ID</code> 變更為專案 ID，並將 <code>CLIENT_ID</code> 變更為您在「取得用戶端 ID」步驟中儲存的用戶端 ID。但請先不要點選「建立」，因為還有幾項資訊需要填寫！	<pre> 36 37 // The project that holds your function 38 const PROJECT_ID = 'my-project'; 39 // Navigate to your webserver's login page and get this from the page. 40 const CLIENT_ID = '123456789.apps.googleusercontent.com'; 41 // This should be part of your webserver's URL: 42 // {tenant-project-id}.appspot.com 43 const WEBSERVER_ID = 'abc8def7ghi6-tp'; 44 // The name of the DAG you wish to trigger 45 const DAG_NAME = 'composer_sample_trigger_response_dag'; 46 </pre>

在 Cloud Shell 中執行下列指令，將 `<your-environment-name>` 替換為 Composer 環境的名稱，並將 `<your-composer-region>` 替換為 Composer 環境所在的區域。

```
gcloud composer environments describe <your-environment-name> --location <your-composer-region>
```

舉例來說，如果您的環境名為 `my-composer-environment`，且位於 `us-central1`，則指令會是

```
gcloud composer environments describe my-composer-environment --location us-central1
```

輸出內容應如下所示：

```

config:
  airflowUri: https://abc123efghi456k-tp.appspot.com
  dagGcsPrefix: gs://narnia-north1-test-codelab-jklmno-bucket/dags
  gkeCluster: projects/a-project/zones/narnia-north1-b/clusters/narnia-north1-test-codelab-jklmno-gke
  nodeConfig:
    diskSizeGb: 100
    location: projects/a-project/zones/narnia-north1-b
    machineType: projects/a-project/zones/narnia-north1-b/machineTypes/n1-standard-1
    network: projects/a-project/global/networks/default
    oauthScopes:
      - https://www.googleapis.com/auth/cloud-platform
    serviceAccount: 987665432-compute@developer.gserviceaccount.com
  nodeCount: 3
  softwareConfig:
    imageVersion: composer-1.7.0-airflow-1.10.0
    pythonVersion: '2'
  createTime: '2019-05-29T09:41:27.919Z'
  name: projects/a-project/locations/narnia-north1/environments/my-composer-environment
  state: RUNNING
  updateTime: '2019-05-29T09:56:29.969Z'
  uuid: 123456-7890-9876-543-210123456

```

在輸出內容中，找出名為 <code>airflowUri</code> 的變數。在 <code>index.js</code> 程式碼中，將 <code>WEBSERVER_ID</code> 變更為 Airflow 網頁伺服器 ID，也就是 <code>airflowUri</code> 變數的一部分，結尾會是「-tp」，例如 <code>abc123efghi456k-tp</code>	<pre>airflowUri: https://abc123efghi456k-tp.appspot.com</pre>
按一下「更多」下拉式選單連結，然後選擇最靠近您的地理區域	Region ? us-central1 ▼
勾選「失敗時重新執行」	☒ Retry on failure ? Warning: your function will retry on failure for up to seven days even if it is due to a bug in your code.
按一下「建立」建立 Cloud 函式	Create Cancel

逐步執行程式碼

從 `index.js` 複製的程式碼大致如下：

```

// [START composer_trigger]
'use strict';

const fetch = require('node-fetch');
const FormData = require('form-data');

/**
 * Triggered from a message on a Cloud Storage bucket.
 *
 * IAP authorization based on:
 * https://stackoverflow.com/questions/45787676/how-to-authenticate-google-cloud-functions-for-access-to-secure-app-engine-endpo
 * and
 * https://cloud.google.com/iap/docs/authentication-howto
 *
 * @param {!Object} data The Cloud Functions event data.
 * @returns {Promise}
 */
exports.triggerDag = async data => {
  // Fill in your Composer environment information here.

  // The project that holds your function
  const PROJECT_ID = 'your-project-id';
  // Navigate to your webserver's login page and get this from the URL
  const CLIENT_ID = 'your-iap-client-id';
  // This should be part of your webserver's URL:
  // {tenant-project-id}.appspot.com
  const WEBSERVER_ID = 'your-tenant-project-id';
  // The name of the DAG you wish to trigger
  const DAG_NAME = 'composer_sample_trigger_response_dag';

  // Other constants
  const WEBSERVER_URL =
`https://${WEBSERVER_ID}.appspot.com/api/experimental/dags/${DAG_NAME}/dag_runs`;
  const USER_AGENT = 'gcf-event-trigger';
  const BODY = {conf: JSON.stringify(data)};

  // Make the request
  try {
    const iap = await authorizeIap(CLIENT_ID, PROJECT_ID, USER_AGENT);

    return makeIapPostRequest(
      WEBSERVER_URL,
      BODY,
      iap.idToken,
      USER_AGENT,
      iap.jwt
    );
  } catch (err) {
    throw new Error(err);
  }
};

```

```

/**
 * @param {string} clientId The client id associated with the Composer webserver application.
 * @param {string} projectId The id for the project containing the Cloud Function.
 * @param {string} userAgent The user agent string which will be provided with the webserver
request.
 */
const authorizeIap = async (clientId, projectId, userAgent) => {
  const SERVICE_ACCOUNT = `${projectId}@appspot.gserviceaccount.com`;
  const JWT_HEADER = Buffer.from(
    JSON.stringify({alg: 'RS256', typ: 'JWT'})
  ).toString('base64');

  let jwt = '';
  let jwtClaimset = '';

  // Obtain an OAuth2 access token for the appspot service account
  const res = await fetch(
    `http://metadata.google.internal/computeMetadata/v1/instance/service-
accounts/${SERVICE_ACCOUNT}/token`,
    {
      headers: {'User-Agent': userAgent, 'Metadata-Flavor': 'Google'},
    }
  );
  const tokenResponse = await res.json();
  if (tokenResponse.error) {
    return Promise.reject(tokenResponse.error);
  }

  const accessToken = tokenResponse.access_token;
  const iat = Math.floor(new Date().getTime() / 1000);
  const claims = {
    iss: SERVICE_ACCOUNT,
    aud: 'https://www.googleapis.com/oauth2/v4/token',
    iat: iat,
    exp: iat + 60,
    target_audience: clientId,
  };
  jwtClaimset = Buffer.from(JSON.stringify(claims)).toString('base64');
  const toSign = [JWT_HEADER, jwtClaimset].join('.');

  const blob = await fetch(
    `https://iam.googleapis.com/v1/projects/${projectId}/serviceAccounts/${SERVICE_ACCOUNT}:signB
lob`,
    {
      method: 'POST',
      body: JSON.stringify({
        bytesToSign: Buffer.from(toSign).toString('base64'),
      }),
      headers: {
        'User-Agent': userAgent,

```

```

        Authorization: `Bearer ${accessToken}`,
    },
}
);
const blobJson = await blob.json();
if (blobJson.error) {
    return Promise.reject(blobJson.error);
}

// Request service account signature on header and claimset
const jwtSignature = blobJson.signature;
jwt = [JWT_HEADER, jwtClaimset, jwtSignature].join('.');
const form = new FormData();
form.append('grant_type', 'urn:ietf:params:oauth:grant-type:jwt-bearer');
form.append('assertion', jwt);

const token = await fetch('https://www.googleapis.com/oauth2/v4/token', {
    method: 'POST',
    body: form,
});
const tokenJson = await token.json();
if (tokenJson.error) {
    return Promise.reject(tokenJson.error);
}

return {
    jwt: jwt,
    idToken: tokenJson.id_token,
};
};

/**
 * @param {string} url The url that the post request targets.
 * @param {string} body The body of the post request.
 * @param {string} idToken Bearer token used to authorize the iap request.
 * @param {string} userAgent The user agent to identify the requester.
 */
const makeIapPostRequest = async (url, body, idToken, userAgent) => {
    const res = await fetch(url, {
        method: 'POST',
        headers: {
            'User-Agent': userAgent,
            Authorization: `Bearer ${idToken}`,
        },
        body: JSON.stringify(body),
    });
    if (!res.ok) {
        const err = await res.text();
        throw new Error(err);
    }
}

```



```
};  
// [END composer_trigger]
```

注意：您的程式碼可能與這裡列印的程式碼略有不同，這沒關係！您在 Cloud Function 中執行的版本是最新版本。

讓我們看看發生了什麼事。這裡有三個函式：`triggerDag`、`authorizeIap` 和 `makeIapPostRequest`

`triggerDag` 是指我們將內容上傳至指定 Cloud Storage bucket 時觸發的函式。我們會在其中設定其他要求中使用的重要變數，例如 `PROJECT_ID`、`CLIENT_ID`、`WEBSERVER_ID` 和 `DAG_NAME`。它會呼叫 `authorizeIap` 和 `makeIapPostRequest`。

```
exports.triggerDag = async data => {  
  // Fill in your Composer environment information here.  
  
  // The project that holds your function  
  const PROJECT_ID = 'your-project-id';  
  // Navigate to your webserver's login page and get this from the URL  
  const CLIENT_ID = 'your-iap-client-id';  
  // This should be part of your webserver's URL:  
  // {tenant-project-id}.appspot.com  
  const WEBSERVER_ID = 'your-tenant-project-id';  
  // The name of the DAG you wish to trigger  
  const DAG_NAME = 'composer_sample_trigger_response_dag';  
  
  // Other constants  
  const WEBSERVER_URL =  
`https://${WEBSERVER_ID}.appspot.com/api/experimental/dags/${DAG_NAME}/dag_runs`;  
  const USER_AGENT = 'gcf-event-trigger';  
  const BODY = {conf: JSON.stringify(data)};  
  
  // Make the request  
  try {  
    const iap = await authorizeIap(CLIENT_ID, PROJECT_ID, USER_AGENT);  
  
    return makeIapPostRequest(  
      WEBSERVER_URL,  
      BODY,  
      iap.idToken,  
      USER_AGENT,  
      iap.jwt  
    );  
  } catch (err) {  
    throw new Error(err);  
  }  
};
```

`authorizeIap` 會使用服務帳戶向保護 Airflow 網路伺服器的 Proxy 發出要求，並「交換」JWT 以取得 ID 權杖，用於驗證 `makeIapPostRequest`。

```

const authorizeIap = async (clientId, projectId, userAgent) => {
  const SERVICE_ACCOUNT = `${projectId}@appspot.gserviceaccount.com`;
  const JWT_HEADER = Buffer.from(
    JSON.stringify({alg: 'RS256', typ: 'JWT'})
  ).toString('base64');

  let jwt = '';
  let jwtClaimset = '';

  // Obtain an OAuth2 access token for the appspot service account
  const res = await fetch(
    `http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/${SERVICE_ACCOUNT}/token`,
    {
      headers: {'User-Agent': userAgent, 'Metadata-Flavor': 'Google'},
    }
  );
  const tokenResponse = await res.json();
  if (tokenResponse.error) {
    return Promise.reject(tokenResponse.error);
  }

  const accessToken = tokenResponse.access_token;
  const iat = Math.floor(new Date().getTime() / 1000);
  const claims = {
    iss: SERVICE_ACCOUNT,
    aud: 'https://www.googleapis.com/oauth2/v4/token',
    iat: iat,
    exp: iat + 60,
    target_audience: clientId,
  };
  jwtClaimset = Buffer.from(JSON.stringify(claims)).toString('base64');
  const toSign = [JWT_HEADER, jwtClaimset].join('.');

  const blob = await fetch(
    `https://iam.googleapis.com/v1/projects/${projectId}/serviceAccounts/${SERVICE_ACCOUNT}:signBlob`,
    {
      method: 'POST',
      body: JSON.stringify({
        bytesToSign: Buffer.from(toSign).toString('base64'),
      }),
      headers: {
        'User-Agent': userAgent,
        Authorization: `Bearer ${accessToken}`,
      },
    }
  );
  const blobJson = await blob.json();
  if (blobJson.error) {
    return Promise.reject(blobJson.error);
  }
}

```

```

}

// Request service account signature on header and claimset
const jwtSignature = blobJson.signature;
jwt = [JWT_HEADER, jwtClaimset, jwtSignature].join('.');
const form = new FormData();
form.append('grant_type', 'urn:ietf:params:oauth:grant-type:jwt-bearer');
form.append('assertion', jwt);

const token = await fetch('https://www.googleapis.com/oauth2/v4/token', {
  method: 'POST',
  body: form,
});
const tokenJson = await token.json();
if (tokenJson.error) {
  return Promise.reject(tokenJson.error);
}

return {
  jwt: jwt,
  idToken: tokenJson.id_token,
};
};

```

`makeIapPostRequest` 會呼叫 Airflow 網路伺服器，觸發 `composer_sample_trigger_response_dag`。DAG 名稱會嵌入透過 `url` 參數傳遞的 Airflow 網路伺服器網址中，而 `idToken` 是我們在 `authorizeIap` 要求中取得的權杖。

```

const makeIapPostRequest = async (url, body, idToken, userAgent) => {
  const res = await fetch(url, {
    method: 'POST',
    headers: {
      'User-Agent': userAgent,
      Authorization: `Bearer ${idToken}`,
    },
    body: JSON.stringify(body),
  });

  if (!res.ok) {
    const err = await res.text();
    throw new Error(err);
  }
};

```

步驟 5 · 約 5 分鐘

5. 設定 DAG

在 Cloud Shell 中，切換至含有範例工作流程的目錄。這是在「取得用戶端 ID」步驟中，從 GitHub 下載的 python-docs-samples 專案。

```
cd
cd python-docs-samples/composer/workflows
```

將 DAG 上傳至 Composer

使用下列指令將範例 DAG 上傳至 Composer 環境的 DAG 儲存空間 bucket，其中 `<environment_name>` 是 Composer 環境的名稱，`<location>` 則是該環境所在的區域名稱。`trigger_response_dag.py` 是我們將使用的 DAG。

```
gcloud composer environments storage dags import \
  --environment <environment_name> \
  --location <location> \
  --source trigger_response_dag.py
```

舉例來說，如果您的 Composer 環境名為 `my-composer`，且位於 `us-central1`，則指令會是

```
gcloud composer environments storage dags import \
  --environment my-composer \
  --location us-central1 \
  --source trigger_response_dag.py
```

逐步執行 DAG

`trigger_response.py` 中的 DAG 程式碼如下所示

```

import datetime
import airflow
from airflow.operators import bash_operator

default_args = {
    'owner': 'Composer Example',
    'depends_on_past': False,
    'email': [''],
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': datetime.timedelta(minutes=5),
    'start_date': datetime.datetime(2017, 1, 1),
}

with airflow.DAG(
    'composer_sample_trigger_response_dag',
    default_args=default_args,
    # Not scheduled, trigger only
    schedule_interval=None) as dag:

    # Print the dag_run's configuration, which includes information about the
    # Cloud Storage object change.
    print_gcs_info = bash_operator.BashOperator(
        task_id='print_gcs_info', bash_command='echo {{ dag_run.conf }}')

```

`default_args` 區段包含 Apache Airflow 中 [BaseOperator](#) 模型所需的預設引數。您會在任何 Apache Airflow DAG 中看到這個部分和這些參數。 `owner` 目前設為 `Composer Example`，但您可以視需要變更為自己的名稱。 `depends_on_past` 顯示這個 DAG 不依附於任何先前的 DAG。這三個電子郵件區段 (`email`、`email_on_failure` 和 `email_on_retry`) 的設定，是根據這個 DAG 的狀態，決定是否要傳送電子郵件通知。由於 `retries` 設為 1，DAG 只會重試一次，且會根據 `retry_delay` 在五分鐘後重試。 `start_date` 通常會與 `schedule_interval` (稍後設定) 搭配使用，決定 DAG 的執行時間，但這個 DAG 的情況並非如此。這項設定為 2017 年 1 月 1 日，但可以設為任何過去的日子。

```

default_args = {
    'owner': 'Composer Example',
    'depends_on_past': False,
    'email': [''],
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': datetime.timedelta(minutes=5),
    'start_date': datetime.datetime(2017, 1, 1),
}

```

注意：在此情況下，不需要變更任何參數，但如果變更了任何參數，請務必重新執行先前的 `gcloud` 匯入指令。

`with airflow.DAG` 區段會設定要執行的 DAG。這項作業會使用工作 ID `composer_sample_trigger_response_dag`、`default_args` 區段的預設引數，以及最重要的 `schedule_interval` `None` 執行。由於我們是透過 Cloud Function 觸發這個特定 DAG，因此 `schedule_interval` 會設為 `None`。因此 `default_args` 中的 `start_date` 並不相關。

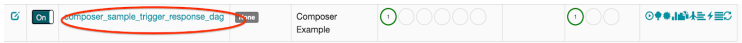
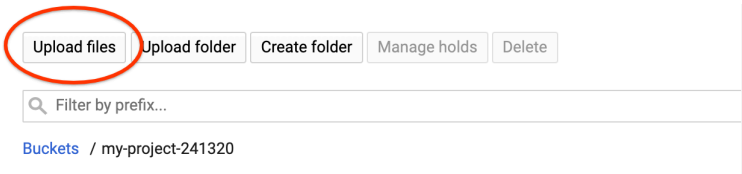
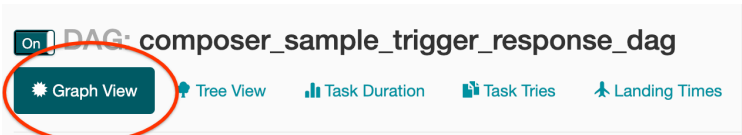
執行時，DAG 會根據 `print_gcs_info` 變數的指示列印設定。

```
with airflow.DAG(
    'composer_sample_trigger_response_dag',
    default_args=default_args,
    # Not scheduled, trigger only
    schedule_interval=None) as dag:

    # Print the dag_run's configuration, which includes information about the
    # Cloud Storage object change.
    print_gcs_info = bash_operator.BashOperator(
        task_id='print_gcs_info', bash_command='echo {{ dag_run.conf }}')
```

重要事項：GitHub 上有這個程式碼的最新版本，請務必使用 `gcloud` 指令複製，而不是從這個程式碼片段複製並貼上。

6. 測試函式

開啟 Composer 環境，然後在環境名稱的資料列中，按一下 Airflow 連結	
按一下 <code>composer_sample_trigger_response_dag</code> 的名稱，開啟該項目。目前不會有任何 DAG 執行記錄，因為我們尚未觸發 DAG 執行。如果這個 DAG 無法顯示或點選，請稍候片刻並重新整理頁面。	
開啟另一個分頁，將任意檔案上傳至先前建立的 Cloud Storage bucket，並指定為 Cloud Function 的觸發條件。您可以透過控制台或使用 <code>gsutil</code> 指令執行這項操作。	
返回 Airflow 使用者界面的分頁，然後按一下「Graph View」	
按一下以綠色外框標示的 <code>print_gcs_info</code> 工作	

按一下選單右上方的「查看記錄」

print_gcs_info on 2019-05-28T07:09:30

Task Instance Details

Rendered

Task Instances

View Log

Run

Ignore All Deps

Ignore Task State

Ignore Task Deps

Clear

Past

Future

Upstream

Downstream

Recursive

Mark Success

Past

Future

Upstream

Downstream

Close

在記錄中，您會看到上傳至 Cloud Storage 值區的檔案
相關資訊。

Log by attempts

1

*** Reading remote log from gcs://us-central1-my-composer-env-7a905919-bucket/logs/composer_sample_trigger_response_dag/print_gcs_info/2019-05-28T07:09:30
[2019-05-28 07:09:36,214] {cli.py:374} INFO - Running on host airflow-worker-6b878b4ff9-569wt
[2019-05-28 07:09:36,289] {models.py:1196} INFO - Dependencies all met for <TaskInstance: composer_sample_trigger_response_dag.print_gcs_info 2019-05-28T07:09:36.301 [models.py:1196] INFO - Dependencies all met for <TaskInstance: composer_sample_trigger_response_dag.print_gcs_info 2019-05-28T07:09:36.302 [models.py:1486] INFO -

Starting attempt 1 of

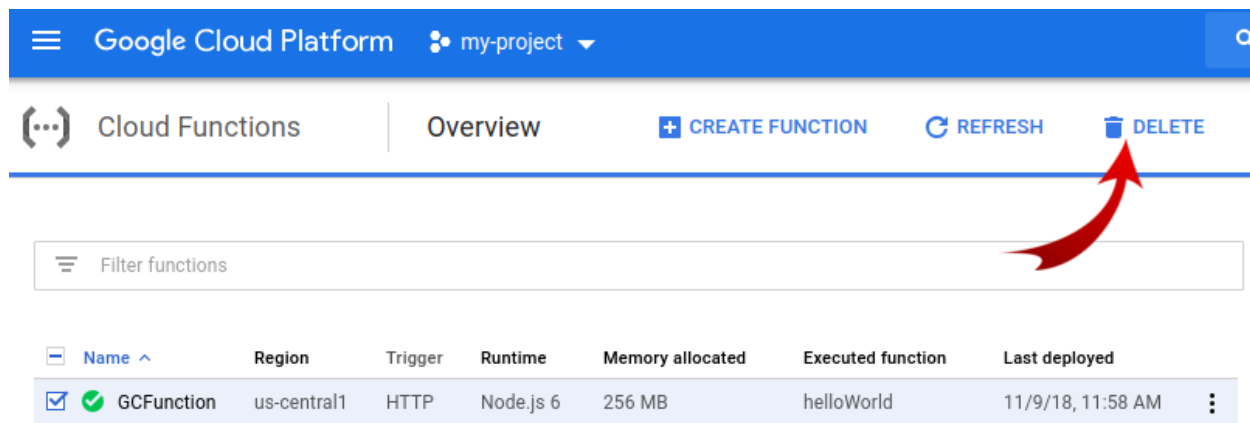
[2019-05-28 07:09:36,368] {models.py:1427} INFO - Executing <Task(BashOperator): print_gcs_info> on 2019-05-28 07:09:30
[2019-05-28 07:09:36,369] {base_task_runner.py:115} INFO - Running: ['bash', '-c', 'u'airflow run composer_sample_trigger_response_dag print_gcs_info 2
[2019-05-28 07:09:38,018] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:38,018] {driver.py:120} INFO - Generating grammar tables from /us
[2019-05-28 07:09:38,041] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:38,041] {driver.py:120} INFO - Generating grammar tables from /us
[2019-05-28 07:09:38,752] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:38,751] {__init__.py:45} INFO - Using executor CeleryExecutor
[2019-05-28 07:09:38,782] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:38,782] {configuration.py:389} INFO - Reading the config from /et
[2019-05-28 07:09:38,802] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:38,802] {app.py:43} WARNING - Using default Composer Environment
[2019-05-28 07:09:39,637] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:39,632] {models.py:189} INFO - Filling up the DagBag from /home/a
[2019-05-28 07:09:39,339] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:39,336] {bash_operator.py:78} INFO - Tmp dir root location:
[2019-05-28 07:09:39,358] {base_task_runner.py:98} INFO - Subtask: /tmp
[2019-05-28 07:09:39,369] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:39,367] {bash_operator.py:88} INFO - Temporary script location: /
[2019-05-28 07:09:39,375] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:39,373] {bash_operator.py:88} INFO - Running command: echo {o'kin
[2019-05-28 07:09:39,419] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:39,413] {bash_operator.py:97} INFO - Output:
[2019-05-28 07:09:39,442] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:39,436] {bash_operator.py:181} INFO - {ukind: ustorage#object, uc
[2019-05-28 07:09:39,949] {base_task_runner.py:98} INFO - Subtask: [2019-05-28 07:09:39,943] {bash_operator.py:185} INFO - Command exited with return

恭喜！您剛才使用 Node.js 和 Google Cloud Functions 觸發了 Airflow DAG ！

7. 清除所用資源

如要避免系統向您的 GCP 帳戶收取您在本快速入門導覽課程中所用資源的相關費用，請按照下列步驟操作：

1. (選用) 如要儲存資料，請從 Cloud Composer 環境的 Cloud Storage bucket，以及您為本快速入門導覽課程建立的儲存空間 bucket [下載資料](#)。
2. 刪除您建立的環境 [Cloud Storage bucket](#)
3. 刪除 [Cloud Composer 環境](#)。請注意，刪除環境並不會刪除環境的儲存空間值區。
4. (選用) 使用無伺服器運算時，每月前 200 萬次叫用免費，且將函式縮放至零時不會產生費用 (詳情請參閱[定價](#))。不過，如要刪除 Cloud Function，請按一下函式總覽頁面右上方的「DELETE」(刪除)



您也可以選擇刪除專案：

1. 前往 GCP 主控台的「Projects」(專案) 頁面。
2. 在專案清單中選取要刪除的專案，然後點按「Delete」(刪除)。
3. 在方塊中輸入專案 ID，然後按一下「Shut down」(關閉) 刪除專案。

警告：刪除專案會出現以下結果：

- **專案中的所有內容都會遭到刪除。**如果您之前使用現有的專案來進行本教學課程，當您刪除該專案時，也會一併刪除您在該專案中完成的所有其他工作。
- **自訂專案 ID 不復存在。**您建立這個專案時，可能已建立日後使用的自訂專案 ID。如要保留使用該專案 ID 的網址 (例如 [appspot.com](#) 網址)，請刪除在該專案中選取的資源，而不是刪除整個專案。